BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

LESSON 249 · Introduction to time and space complexity

Name: _____ Class: _____ Date: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

Do Now · Recall previous learning

Q1. What makes a binary search trace convincing?

Q2. What is its typical average time complexity?

Q3. How many values are in an int array of length n?

Q4. Apply an eight-bit left shift by two to 00010110_2 and bitwise AND to 00110110_2 with 00001111_2. Give both results in binary.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

I Do · Read new knowledge

Key vocabulary

time complexity — Asymptotic growth in algorithm work.

auxiliary space — Extra memory used by an algorithm.

Big O — Upper asymptotic bound commonly used to classify growth.

quadratic — Growth proportional to n^2.

exponential — Growth such as 2^n.

New learning

Analyse the loops — One full loop runs n iterations; a pair of n-sized nested loops performs n^2 inner iterations.

Separate time and space — Big O time counts growth of work; auxiliary space measures extra storage beyond the original input.

Compare feasible scales — At n=16, exponential growth can be dramatically more expensive than linear growth. Big O omits constants and measured run time.

Worked example

For n=8, compare one loop visiting each item with two full nested loops.

1. Predict first — One scan has eight iterations, giving linear growth O(n).

2. Explain the answer — Nested loops perform 8×8=64 inner iterations, giving quadratic growth $O(n^2)$.

We Do · Practise with guidance

For n=16, compare linear and quadratic operation counts. What happens when n doubles from 8?

```
linear = n; quadratic = n*n;
// For n=16: 16 and 256
```

Predict first

The linear count doubles from 8 to 16.

Explain the answer

Quadratic count rises from 64 to 256, multiplying by four; this difference grows with n.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

Question 1

State the Big O of (a) constant assignment, (b) one n-element loop, (c) two full nested n-loops and (d) repeatedly halving an interval.

Question 2

A program copies an n-element array into a new n-element array. Give its time and auxiliary-space complexity and justify both.

Question 3

A method uses an in-place $O(n^2)$ sort and another uses merge sort with O(n log n) time and O(n) auxiliary space. Explain one situation favouring each.

Exit Ticket · Answer before checking

O(n) time necessarily means an algorithm uses O(n) additional memory.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

Page 1 onwards · Do Now

Q1. Showing low, high, mid, comparisons and any bound updates.

Q2. O(n log n).

Q3. n values; storing it needs O(n) space.

Q4. Left shift 00010110 << 2 = 01011000 (eight-bit, zero-filled). AND 00110110 ∧ 00001111 = 00000110.

You Do pages · Independent questions

Q1. [4] O(1) [1], O(n) [1], O(n^2) [1], O(log n) [1].

Q2. [6] Each of n elements read and written [2], so O(n) time [1]; allocates n extra positions [2], so O(n) auxiliary space [1].

Q3. [6] Merge sort can scale better to large n [2]; O(n log n) vs O(n^2) [1]; in-place sort can suit small data or tight memory [1]; typically O(1) auxiliary space for in-place quadratic sorting [1]; compare time-space trade-off [1].

Exit Ticket · Model answer

False. Time and auxiliary-space complexity are different measurements.

Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

1. Copy the whole original question:

2. Write a complete new answer in GREEN PEN. Show working if needed:

BLACK PEN Answer all questions first.

GREEN PEN Then: tick and rewrite mistakes.

Do Now Recall previous learning | **I Do** Read new knowledge | **We Do** Practise with guidance | **You Do** Work independently | **Review & Improve** Check understanding